When Can Off-the-Shelf LLMs Classify the GPU Roofline? The Role of Source- versus Compilation-Level Evidence

Anonymous Author(s)
Anonymous Institution(s)

***Abstract*—AI coding agents increasingly propose GPU code changes, but evaluating those changes still depends on compile-run-profile loops on scarce target hardware. We study a software-engineering question that sits earlier in that loop: before any profiling run, can off-the-shelf large language models decide whether a GPU kernel is bandwidth-bound or compute-bound on a target device, and what evidence do they need to do it?**

We cast the task as predicting profiler-derived DRAM-level Roofline behavior from software artifacts. For each first kernel invocation, models predict launch dimensions, FP16/FP32/FP64 floating-point work, and DRAM bytes read and written; we derive Roofline Arithmetic Intensity (RAI) and the bandwidth-bound versus compute-bound (BB/CB) Roofline regime from those quantities. We evaluate 254 CUDA and OpenMP kernels (95 programs, 1,016 profiled samples) across four NVIDIA GPUs, comparing *source-only* prompts against prompts that add post-compilation SASS, which is obtainable by cross-compilation without target hardware.

Our central finding is about evidence, not raw accuracy. Source code alone is enough for the frontier models to classify the FP32 Roofline regime well (*source-only* balanced accuracy 0.940 for `Opus 4.6`), but it is essentially blind for half precision, where every model stays at the 0.500 chance baseline. The reason is mechanistic: the decisive FP16 arithmetic is compiler-synthesized (e.g., `HFMA2`) and invisible in source. Adding cross-compiled SASS recovers it, lifting FP16 BB/CB balanced accuracy from 0.500 to 0.984 for `Opus 4.6` and 0.870 for `GPT-5.4`. These classification signals are cheap: a budget open model classifies the *source-only* FP32 regime at 0.875 balanced accuracy for roughly $0.0006 per kernel, orders of magnitude below a profiling run, while frontier+SASS is a premium tier that additionally cracks FP16. Numeric RAI itself is accurate only in identifiable regimes and is best read as a diagnostic for *why* classification succeeds or fails; lightweight static baselines recover part of the same signal and win in a few regimes, so the deployable shape is a cost-tiered cascade rather than an LLM-only oracle. We package the benchmark—source, build context, runtime arguments, SASS, profiler-derived labels, and model outputs—as a reusable artifact for execution-free GPU performance reasoning; public URLs are omitted in this anonymous version. Our claim is bounded: off-the-shelf LLMs can classify the Roofline regime of a first kernel invocation within identifiable, cheaply-detectable evidence tiers, but they do not replace profiling.

***Index Terms*—software performance, GPUs, Roofline, arithmetic intensity, large language models, static performance reasoning**

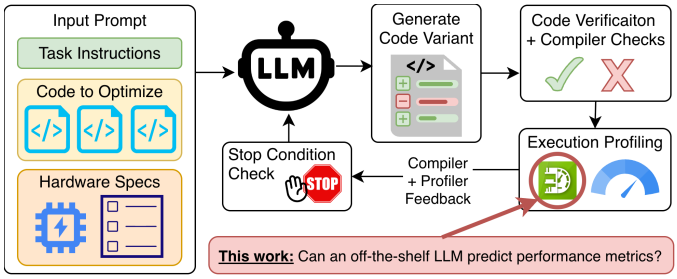

Fig. 1. Profiler-in-the-loop GPU optimization is effective, but it assumes repeated access to the target accelerator. We study whether off-the-shelf LLMs can classify a kernel's bandwidth-bound versus compute-bound Roofline regime earlier in that loop, from software artifacts alone, before runtime profiling is available.

I. Introduction

GPU performance engineering is still fundamentally measurement-driven. Developers usually optimize kernels by compiling candidate variants, running them on the target accelerator, profiling hardware counters, and then deciding whether to focus on computation, memory traffic, or both. That workflow is already costly for humans, and it becomes even more expensive in agentic software-engineering loops where an LLM proposes many candidate edits and expects profiler feedback after each one [1]–[4].

In practice, target-GPU access is often the bottleneck. Accelerators may be rationed on a shared cluster, unavailable in early development, blocked by missing system dependencies, or simply too expensive to occupy for speculative triage. The practical question is therefore not "can an LLM replace a profiler?" but a narrower one that a developer or coding agent faces *before* a profiling run is available: on a given target GPU, is this kernel likely to be limited by data movement or by computation?

That bandwidth-bound versus compute-bound (BB/CB) decision is the first branch of Roofline

analysis [5], and it is the one that determines which optimization direction is even worth attempting. It is also fundamentally a *classification*, not a precise number: a developer who learns that a kernel is firmly compute-bound can act on that without knowing its arithmetic intensity to two decimals. We therefore make Roofline-regime classification the primary object of study, and treat the underlying Roofline Arithmetic Intensity (RAI) as a diagnostic that explains *why* a classification is right or wrong rather than as the headline deliverable.

We study three off-the-shelf LLMs in two evidence settings. The *source-only* setting provides only pre-compilation artifacts—source files, build commands, runtime arguments, and GPU specifications—and corresponds to early development, code review, or agent planning. The *source+SASS* setting adds the target kernel's SASS disassembly. Crucially, SASS is obtained by *cross-compilation* (for example, `nvcc -arch=sm_90` followed by `cuobjdump -sass`) and therefore requires no target hardware; both settings are execution-free at prediction time. Comparing them isolates what compiler lowering adds over source-visible semantics.

We organize the study around three research questions:

- **RQ1.** Can off-the-shelf LLMs classify a kernel's BB/CB Roofline regime from source artifacts alone, and for which precisions?
- **RQ2.** What does cross-compiled SASS add to that classification, and why?
- **RQ3.** How do the resulting signals compare—in accuracy and in cost—against lightweight static baselines and against profiling itself?

To answer them we build on 95 HeCBench CUDA and OpenMP programs. They expose 254 first-invocation GPU kernels and 1,016 profiled samples across an RTX 3080, A10, A100, and H100. For each sample the models predict launch dimensions, per-precision FP16/FP32/FP64 floating-point work, and DRAM bytes read and written; we derive both RAI and its BB/CB side relative to each GPU's Roofline balance point. For fair cross-model comparison we use the 732 samples completed by all three models in both evidence settings.

Three findings matter most.

First (RQ1), source artifacts already support good single-precision regime classification but are blind to half precision. `Opus 4.6` classifies the *source-only* FP32 regime at 0.940 nonzero balanced accuracy (and FP64 at 0.755), and even the budget `GPT OSS` reaches 0.875 for FP32. For FP16, however, all three models sit at the 0.500 chance baseline from source alone.

Second (RQ2), the FP16 gap is an evidence gap, not a model-capability gap, and it closes with compilation. The decisive half-precision arithmetic is compiler-synthesized—packed `HFMA2` operations that are not syntactically present in source—so source-only reasoning cannot see it. Adding cross-compiled SASS lifts FP16 BB/CB balanced accuracy from 0.500 to 0.984 for `Opus 4.6` and 0.870 for `GPT-5.4`. This is the paper's clearest result: *compilation, not execution, is what half-precision Roofline reasoning requires*, and compilation is available without the target GPU.

Third (RQ3), these signals are cheap relative to profiling, and they form a tier. A *source-only* FP32 regime classification from the budget model costs on the order of $0.0006 per kernel—orders of magnitude below occupying a target GPU for a profiling run—while the frontier+SASS configuration is a premium tier that additionally resolves FP16. Lightweight deterministic and learned static baselines recover part of the same signal and even win in a few regimes (notably *source+SASS* FP64), so the deployable shape is a cost-tiered cascade among execution-free strategies rather than an LLM-only oracle. Numeric RAI estimation is accurate only in identifiable regimes and is best used as a diagnostic and a coarse profiling-priority hint, not as a runtime substitute.

Our contribution is squarely about software engineering. We study how execution-relevant software artifacts can support AI-assisted performance reasoning before execution.

More concretely, we make four contributions:

- We formulate execution-free Roofline-regime classification as a software-engineering task for early GPU-kernel triage, with software artifacts as inputs and profiler-derived BB/CB labels as ground truth.
- We show that the source-versus-compilation evidence boundary, not model choice, governs half-precision triage, and give the mechanistic compiler-synthesized-FP16 explanation.
- We quantify the accuracy/cost tiering of off-the-shelf prompting against context-only, deterministic, and lightweight learned static baselines, and against the cost of profiling.
- We package the benchmark—source, build context, runtime arguments, SASS, profiler-derived labels, and model outputs—as a reusable artifact for execution-free GPU performance reasoning, documenting limits that future work can address with repeated-query and reasoning-versus-memorization studies.

The remainder is organized as follows. Section II positions our work relative to Roofline analysis, execution-free GPU performance prediction, and LLM-based optimization. Sections III and IV define the prediction task and study design. Section V answers the research questions. Sections VI and VII translate the findings into implications and limitations for ICSE readers, and Section VIII concludes.

II. Background and Related Work

We sit at the intersection of Roofline-guided GPU performance engineering, execution-free performance prediction, and LLM-based software engineering. The relevant prior work is broad, but the distinctions that matter for this study are fairly crisp.

a) Roofline-guided GPU triage.: The Roofline model gives a compact way to reason about whether a kernel is limited primarily by computation or by data movement [5]. Its horizontal axis, arithmetic intensity, is not a full performance model; it is an interpretable ratio that helps developers decide which optimization direction is worth pursuing first. That interpretability is why we use RAI as a triage target rather than a more opaque prediction objective. Our contribution is therefore not another Roofline model, but an empirical study of when LLMs can estimate a Roofline-relevant signal from software artifacts before target execution.

b) Execution-free GPU performance prediction.: Long before LLMs, GPU researchers studied analytical and learned predictors that estimate performance without executing the target kernel. Their inputs include static program structure, microarchitectural assumptions, PTX/SASS features, and compiler-derived summaries [6]–[12]. Related analytical and learned predictors also study transfer across workloads or devices and learned representations such as GNNs or hybrid LLM/GNN predictors. These works establish that execution-free prediction is valuable, but they generally target runtime, throughput, power, energy, or static execution statistics rather than the profiler-derived BB/CB regime we study. They also usually rely on task-specific modeling pipelines, compiler analyses, or learned predictors rather than zero-training prompting. Our question is narrower and more software-engineering-oriented: what can a developer tool obtain from existing code/build artifacts using an unmodified LLM, and where should that tool fall back to cheaper static features or runtime profiling?

This difference in objective also explains why we do not benchmark directly against these predictors, and instead compare against purpose-built static baselines (Section IV). The analytical and learned models above predict a different quantity—execution time [6]–[8], latency [12], power and energy under frequency scaling [9], [10], or static basic-block frequencies [11]—so repurposing them to per-precision DRAM-level Roofline classification would require re-implementing them against a target they were never designed or validated for, yielding a comparison that reflects our re-implementation rather than the published method. Several also assume inputs outside our setting (PTX feature pipelines, fitted hardware-independent features) or target older architectures with no artifact retargetable to Ampere/Hopper, and most are CUDA- or OpenCL-only and so cannot address the OpenMP-offload half of our corpus. Where the prior work is a *learned* predictor over compiler-IR features [9], [10], [12], we instead instantiate that class faithfully for our target with the grouped-cross-validated random-forest and extra-trees baselines of Section IV, rather than mis-running an artifact built for a different objective.

c) LLMs for GPU optimization and code reasoning.: Recent work has shown that LLMs can generate or optimize accelerator code, especially when compilation, execution, testing, or profiling is available inside the loop [2]–[4], [13]. Other work studies compiler-oriented representations and accelerator-code benchmark suites. That literature is directly relevant to ICSE because it frames GPU tuning as an AI-assisted software-development activity. Our study complements those systems rather than competing with them. We do not optimize kernels directly; we study whether an LLM can provide an earlier triage signal when the profiler loop itself is unavailable or too costly.

d) LLMs for performance prediction.: The closest neighboring work asks LLMs to predict GPU metrics directly, and the nearest two are both *fine-tuned*. LLM-Perf fine-tunes models for OpenCL kernel performance estimation [14], and Omniwise fine-tunes a model to predict several GPU metrics, including arithmetic intensity, execution-free, reporting over 90% of predictions within 10% [15]. Omniwise is the closest result to ours, and we confront it directly rather than competing with it: it answers a different question (what a *trained* model can predict), it targets AMD MI250/MI300X with no released model retargetable to the NVIDIA GPUs and counters we study, and so it cannot be run on our corpus. Our study is the complementary zero-training counterpart—if anything, Omniwise's strong fine-tuned numbers make the off-the-shelf gap we characterize the interesting question. Prior work by Bolet *et al.* studies coarse BB/CB classification of parallel code [16]; the present work generalizes that precursor from a single coarse label to a per-precision, decomposed Roofline signal and to the source-versus-compilation evidence question. Our contribution therefore differs in three ways. First, we study the zero-training regime with off-the-shelf models rather than fine-tuned predictors. Second, we target a decomposed signal by asking models to predict launch dimensions, per-precision FLOP counts, and DRAM bytes, from which both the regime label and RAI follow. Third, we explicitly compare *source-only* reasoning with prompts that paste target-kernel SASS instructions into the context, isolating the gap between source-visible semantics and compiler-realized behavior. That distinction matters because compiler lowering, instruction selection, and floating-point implementations can introduce work that is not syntactically obvious in source code [17], [18].

e) Static reasoning with code LLMs.: Our task also relates to a broader line of work showing that code LLMs can reason about latent program behavior, but only imperfectly [19]–[23]. Estimating RAI requires more than recognizing syntax: the model must propagate launch parameters, recover thread-level work, account for mixed precision, and reason about DRAM-visible traffic instead of source-level loads and stores. That makes RAI prediction a useful probe of execution-relevant code reasoning in a form that is directly actionable for software engineering.

f) Gap addressed by this study.: Taken together, prior work leaves an open gap. We know that static models

can predict GPU behavior without execution, that LLM optimizers benefit from profiler feedback, and that fine-tuned LLM predictors can estimate GPU metrics. What remains underexplored is whether *off-the-shelf* LLMs can provide a useful execution-free triage signal from ordinary software artifacts alone. We address that gap by studying quantitative RAI prediction, separating source-visible from post-compilation evidence, and characterizing the regimes where off-the-shelf prompting is accurate, merely useful for triage, or unreliable.

III. Task Formulation and Methodology

A. Prediction Task

Our target is the DRAM-level Roofline Arithmetic Intensity (RAI) of a GPU kernel's *first invocation*. At prediction time, the LLM does not execute the program, collect profiler counters, or observe runtime traces. Instead, it receives software artifacts and must infer the quantities from which RAI is derived.

We do not ask the model to emit RAI directly. For each profiled sample, meaning one first-invocation kernel execution on one GPU, the model predicts seven values:

1) grid size,
2) block size,
3) FP16 operation count,
4) FP32 operation count,
5) FP64 operation count,
6) DRAM bytes read, and
7) DRAM bytes written.

We then derive RAI separately for each precision:

$$\texttt{RAI}_p = \frac{\texttt{FLOP}_p}{\texttt{BytesRead} + \texttt{BytesWritten}}, \qquad p \in \{\texttt{FP16}, \texttt{FP32}, \texttt{FP64}\}.$$

This decomposition makes failures easier to interpret. An incorrect launch configuration propagates into total work; an incorrect FLOP count indicates semantic or compiler-lowering errors; and an incorrect byte estimate indicates difficulty reasoning about DRAM-visible traffic.

We intentionally keep separate FP16/FP32/FP64 RAI values rather than collapsing mixed-precision kernels into a single scalar. This matches the per-precision balance points used later for triage and makes compiler-injected FP16 work visible. It also means that a kernel may have zero FP64 RAI but nonzero FP32 RAI, which we treat as a feature of the task rather than a modeling mistake.

B. Input Evidence

Figure 2 summarizes the evidence we provide to the model. Every prompt contains the kernel identifier (mangled and demangled), relevant source files, compile commands, runtime arguments, and target-GPU specifications. The *source+SASS* configuration augments the same prompt with SASS sections for the target kernel and helper routines that may execute on its path.

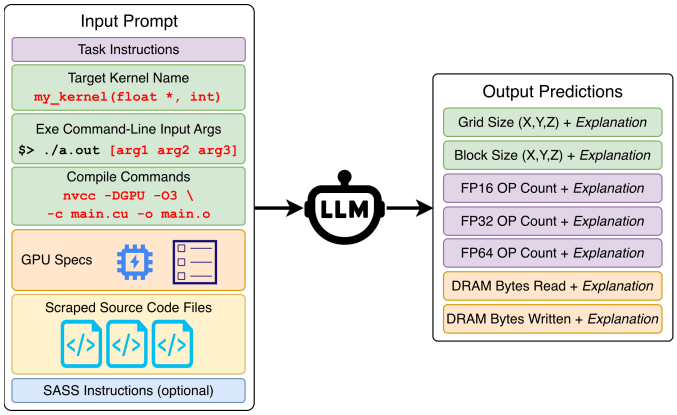

Fig. 2. Per-kernel prompting workflow. Each query includes the target kernel name, source files, compile commands, runtime arguments, and GPU specifications. The *source+SASS* setting adds target-kernel SASS sections and relevant helper routines so we can measure the value of post-compilation evidence.

The distinction between *source-only* and *source+SASS* is important. *source-only* corresponds to early development, code review, or agent planning when only pre-compilation artifacts are available. *source+SASS* still avoids target execution and profiling at prediction time, but it assumes target-specific compilation is possible, which is a materially different stage of the workflow. We therefore treat *source+SASS* as *execution-free* rather than broadly "hardware-free."

The model returns a structured tool-call response containing the seven predicted quantities plus short explanations. Those explanations are useful for sanity checking and for the later exploratory failure analysis, but they are not used to compute the reported metrics.

C. Profiler-Derived Ground Truth

Ground truth comes from an offline benchmark-building process, not from the prediction loop itself. We profile each benchmark with NVIDIA Nsight Compute and retain the first invocation of the target kernel in each run. We repeat the executable three times and average the retained first-invocation counters to reduce profiling noise.

The exact DRAM-traffic counters are dram__bytes_read.sum and dram__bytes_write.sum. Floating-point counts are reconstructed from executed-operation counters:

- FP32:
  smsp__sass_thread_inst_executed_op_fadd_pred_on.sum.per_cycle_elapsed, smsp__sass_thread_inst_executed_op_fmul_pred_on.sum.per_cycle_elapsed, derived__smsp__sass_thread_inst_executed_op_ffma_pred_on_x2.
- FP64:
  smsp__sass_thread_inst_executed_op_dadd_pred_on.sum.per_cycle_elapsed, smsp__sass_thread_inst_executed_op_dmul_pred_on.sum.per_cycle_elapsed, derived__smsp__sass_thread_inst_executed_op_dfma_pred_on_x2.

- FP16:

  smsp__sass_thread_inst_executed_op_hadd_pred_on.sum.per_cycle_elapsed, smsp__sass_thread_inst_executed_op_hmul_pred_on.sum.per_cycle_elapsed, derived__smsp__sass_thread_inst_executed_op_hfma_pred_on_x4.

We convert those per-cycle rates to total operation counts using smsp__cycles_elapsed.avg.per_second and gpu__time_duration.sum.

These choices matter for scope. Because bytes are counted at the DRAM interface, the task asks the model to predict *DRAM-visible* traffic rather than raw source-level memory operations. Writes that remain in the cache hierarchy until later are not immediately visible in the denominator. Likewise, compiler-injected helper routines for division or transcendental functions contribute to the measured FLOP counts even when they are not obvious in source. Those are precisely the cases where *source+SASS* may add value.

D. Why First Invocation and DRAM-Level RAI

We make two deliberate simplifications. First, we evaluate only the first invocation of each kernel, because repeated invocations introduce temporal cache reuse and cross-kernel interference that would substantially widen the reasoning space. Second, we focus on DRAM-level RAI rather than a cache-aware Roofline. This keeps the prediction target tied to one interpretable denominator while still exposing realistic sources of error such as L2 write-back behavior.

These simplifications narrow the claim. The paper studies execution-free early triage for a first kernel invocation at the DRAM level; it does not claim to predict steady-state runtime behavior, cache-resident arithmetic intensity, or end-to-end application performance.

IV. Study Design

A. Benchmark Corpus

We draw workloads from the HeCBench suite [24], focusing on CUDA and OpenMP target-offload programs because they are both common GPU programming models and represent distinct source/build ecosystems [25]. The final corpus contains 95 benchmark programs: 56 CUDA programs and 39 OpenMP programs. Those programs expose 254 first-invocation GPU kernels: 155 CUDA kernels and 99 OpenMP kernels.

Profiling the corpus with NVIDIA Nsight Compute on four NVIDIA GPUs yields 1,016 samples. Each sample induces three precision-specific RAI labels (FP16/FP32/FP64), so the profiler-derived ground-truth space contains 3,048 precision-specific evaluation points before any model-query filtering.

We profile on four GPUs chosen to span different memory and compute regimes: RTX 3080, A10, A100, and H100. Table I lists the hardware characteristics and the resulting DRAM-level balance points used for BB/CB classification. We deliberately target DRAM-level RAI rather than a cache-aware Roofline variant, because DRAM traffic gives one profiler-derived signal that can be compared consistently across all samples. The paper's architectural question is therefore not whether models generalize to all accelerators; it is whether the same prompting strategy remains useful across noticeably different NVIDIA deployment targets.

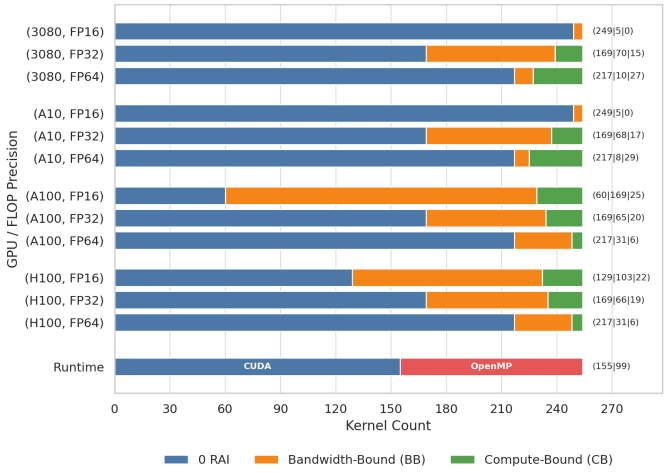

Fig. 3. Ground-truth RAI class distribution by GPU and precision. Zero-valued RAI cases dominate every precision, FP16 nonzero cases are concentrated on A100/H100, and the nonzero class mix varies by device. These imbalances motivate separate evaluation of zero detection, nonzero regression, and nonzero BB/CB triage.

Figure 3 highlights the resulting class imbalance. Across the full 1,016 samples, FP16 has 687 zero cases, FP32 has 676, and FP64 has 868. FP16 is especially architecture-sensitive because the A100/H100 binaries contain many more nonzero FP16 measurements than the 3080/A10. This is precisely why we separate the evaluation tasks instead of reporting a single aggregate error number.

B. Models, Prompts, and Sample Accounting

We study three off-the-shelf LLMs: `GPT-5.4`, `Opus 4.6`, and the cheaper `GPT OSS` baseline. All reported comparisons use the two released prompt configurations that differ only by the presence or absence of SASS: *source-only* and *source+SASS*. We do not fine-tune the models, train task-specific heads, or expose runtime feedback inside the prediction loop. The committed run script invokes one trial per sample for the model identifiers `openai/gpt-5.4`, `anthropic/claude-opus-4.6`, and `openai/gpt-oss-120b`; the released OpenRouter client configuration defaults to temperature 0.2 and top-p 0.1.

The expensive frontier models were queried once per sample in the released evaluation subset. That is a methodological limitation of the artifact, so we report coverage explicitly rather than hiding it inside aggregate metrics. The completed evaluation subset contains:

- 3,048 *source+SASS* and 3,048 *source-only* predictions for `Opus 4.6`,

TABLE I
PROFILED GPUS AND DRAM-LEVEL BALANCE POINTS USED FOR BB/CB CLASSIFICATION.

	RTX 3080	A10	A100	H100
Base architecture	Ampere (GA102)	Ampere (GA102)	Ampere (GA100)	Hopper (GH100)
Compute capability	8.6	8.6	8.0	9.0
Memory bandwidth (GB/s)	760	600	1,555	3,360
L2 cache (MB)	5	6	40	50
FP16 peak (TFLOP/s)	30.55	15.62	77.97	133.82
FP32 peak (TFLOP/s)	30.55	15.62	19.49	66.91
FP64 peak (TFLOP/s)	0.477	0.244	9.75	33.45
FP16 balance point (FLOP/byte)	40.20	26.03	50.14	39.83
FP32 balance point (FLOP/byte)	40.20	26.03	12.53	19.91
FP64 balance point (FLOP/byte)	0.63	0.41	6.27	9.96

TABLE II
GROUND-TRUTH CLASS BALANCE ON THE 732-SAMPLE SHARED SUBSET USED FOR ALL PAIRED CROSS-MODEL COMPARISONS. THE IMBALANCE IS PRECISION-DEPENDENT: FP16/FP32 HAVE MORE NONZERO SAMPLES THAN FP64, BUT ALL THREE PRECISIONS REMAIN ZERO-HEAVY, WHICH IS WHY TASKS A/C/D ARE REPORTED SEPARATELY.

Precision	Zero	Nonzero BB	Nonzero CB	Zero share
FP16	513	185	34	0.701
FP32	496	186	50	0.678
FP64	623	56	53	0.851

TABLE III
HOW TO READ THE EVALUATION METRICS.

Metric	Reader interpretation
≤25% APE	Fraction of nonzero samples where numeric RAI is close enough to treat as an accurate static estimate.
MedAPE	Typical nonzero numeric error; useful for seeing heavy-tailed failures, not the whole story.
BalAcc	Macro recall for imbalanced triage tasks, so zero or bandwidth-heavy classes cannot dominate the score.

- 3,048 *source+SASS* and 3,045 *source-only* predictions for `GPT-5.4`, and
- 2,535 *source+SASS* and 2,574 *source-only* predictions for `GPT OSS`.

`GPT OSS` incompletions primarily reflect context-overflow or completion failures on long prompts, so we report coverage explicitly instead of folding missing samples into aggregate quality metrics. For fair cross-model comparisons, we use the 732 samples completed by all three models in both prompt settings. Within-model source-versus-SASS comparisons are paired on the subset completed by both settings for that model.

The fixup tables are generated read-only from the same artifact products used by the repository reproduction script. The direct-prompting `make_plots_for_paper.py` script reconstructs model predictions from the restored PostgreSQL dump and writes the paper-facing shared-sample outputs; the website mirror `site-data.json` is the derived prediction table used by the fixup generators. The static baselines additionally read the released `dataset-creation/gpuFLOPBench.json`, which contains source links and static instruction-mix data.

C. Evaluation Tasks and Metrics

The evaluation intentionally separates four related but distinct tasks:

a) Task A: zero vs. nonzero detection.: We first ask whether a model can distinguish kernels whose per-precision RAI is zero from kernels with nonzero RAI. Because the obvious trivial baseline is to predict zero everywhere, we report balanced accuracy rather than raw accuracy. Only an exact zero numeric prediction counts as "zero"; any positive or non-finite prediction is conservatively treated as "nonzero."

b) Task B: nonzero RAI regression.: For samples whose ground-truth RAI is nonzero, we report median absolute percent error. The distributions are heavy-tailed, so we emphasize medians and the supporting boxplots rather than means. As a numeric lower bound, we also compare against a trivial global-median-RAI heuristic, which produces approximately 98.5%, 99.9%, and 99.6% median APE for FP16, FP32, and FP64 respectively on the shared subset.

c) Task C: nonzero BB/CB triage.: For nonzero cases, we compare the expected and predicted side of the GPU-specific Roofline balance point. We report balanced accuracy because the class ratios are uneven. The released artifact also exports compute-bound precision/recall/F1 so reviewers can see whether a method reaches high recall by over-predicting the compute side. If a model predicts zero for a truly nonzero kernel, we count that as a memory-side failure and collapse it onto the bandwidth-bound side for this binary triage metric. This is conservative, but appropriate: a zero prediction still fails to identify compute-side behavior.

d) Task D: three-way triage.: Finally, we retain the full {zero, BB, CB } label space to show whether a model preserves the higher-level triage outcome that a developer or agent would see. We report multiclass balanced accu-

racy (macro recall). As in Task C, non-finite predictions are conservatively counted as memory-side failures rather than as a separate abstain class.

We include two trivial class baselines for context. Always predicting zero yields 0.50 balanced accuracy on Task A and 0.33 balanced accuracy on Task D. Always predicting the majority nonzero class (bandwidth-bound in all three precisions here) yields 0.50 balanced accuracy on Task C.

We also add three simple non-LLM reference baselines. The *context median* baseline uses grouped cross-validation and predicts the training-fold median RAI for each runtime×GPU pair, with runtime-only, GPU-only, and global fallbacks when a pair is unseen. This is a deliberately naive context-only baseline that uses no code or disassembly evidence. We also add two deterministic non-LLM reference heuristics derived from the released artifact. The *source lexical* heuristic uses only kernel-linked source files: if no precision-relevant floating-point type or math-library token appears, it predicts zero; otherwise it estimates RAI as approximate arithmetic-token count divided by four times approximate memory-reference count. The *SASS mnemonic* heuristic maps each GPU to the released architecture-specific static instruction mix, counts precision-relevant floating-point arithmetic mnemonics, counts global-memory mnemonics, and converts those counts into an approximate static RAI using 2/4/8-byte denominators for FP16/FP32/FP64. These baselines are intentionally coarse. They ignore loop trip counts, control-flow frequencies, vector widths, and dynamic access sizes, so we use them only as deterministic reference points rather than as optimized competitors.

We additionally add lightweight learned static baselines from two tree families: random forests and extra trees. For each prompt setting and family, we run a five-fold GroupKFold evaluation grouped by program. Stage 1 predicts zero versus nonzero RAI; Stage 2 regresses $\log(1 + RAI)$ on the nonzero training samples and emits zero whenever Stage 1 predicts zero. The *source-only* setting uses only source-derived lexical counts plus runtime and GPU identifiers; the *source+SASS* setting adds architecture-specific static instruction-mix counts derived from the released post-compilation artifact. The generated artifact tables report the seed-0 runs and five-seed sensitivity for the learned baselines. Because these baselines are trained and evaluated only within the released corpus, we use them as lightweight static references rather than as production-quality predictors.

Secondary checks appear only where they answer a specific comparison question. Recall@10% measures fixed-budget profiling utility, exact paired binomial tests summarize discordant Task C wins on the same samples, and Cliff's δ is reserved for exploratory feature-vote error analysis.

TABLE IV
Static reference baselines and their inputs.

Baseline	Inputs and training protocol
Context median	Runtime and GPU only; grouped cross-validation predicts the training-fold median RAI with runtime/GPU fallbacks.
Source lexical	Deterministic source-token counts from `gpuFLOPBench.json`: arithmetic operators, memory references, types, and math calls; no training.
SASS mnemonic	Deterministic architecture-specific instruction-mix counts from `gpuFLOPBench.json`: floating-point and global-memory mnemonics; no dynamic counts.
Learned RF/ET	Random forest or extra-trees Stage 1 zero classifier plus Stage 2 $\log(1 + RAI)$ regressor; GroupKFold by program; *source-only* uses source lexical counts plus runtime/GPU, and *source+SASS* adds static SASS instruction counts.

D. Exploratory Failure Analysis

The released artifact also contains an auxiliary feature-voting pipeline. It uses inexpensive LLMs to label source patterns such as division, special math, reusable subexpressions, and loop-invariant floating-point work. We retain that analysis because it helps characterize where prediction errors cluster, but we explicitly treat it as exploratory. The feature labels are not human-validated in the released data, so we use them to describe repeated failure patterns rather than to make causal claims.

V. Results

Unless stated otherwise, the main cross-model comparisons use the 732 samples completed by all three models in both prompt settings. That shared subset avoids coverage bias from the incomplete `GPT OSS` runs while preserving paired source-versus-SASS comparisons. We report nonzero BB/CB regime classification as the primary outcome and use numeric RAI error as a diagnostic; the underlying zero/nonzero detection and three-way summaries appear in the appendix.

A. RQ1: Can Source Alone Classify the Roofline Regime?

Answer. For single precision, yes; for half precision, no.

Table V reports nonzero BB/CB balanced accuracy by model, precision, and evidence setting. From source alone, `Opus 4.6` classifies the FP32 regime at 0.940 balanced accuracy and FP64 at 0.755; `GPT-5.4` reaches 0.889/0.670, and even the budget `GPT OSS` reaches 0.875/0.690. These are strong numbers for a pre-compilation signal: the models recover the compute-versus-memory branch for most single-precision kernels using only source, build context, runtime arguments, and GPU specifications. The compute-bound precision/recall breakdown (Appendix Table XI) shows this is not an artifact of over-predicting one class: source-only FP32 `Opus 4.6` reaches 1.000 precision at 0.880 recall on the compute-bound side, and 0.966 precision at 0.528 recall for FP64.

TABLE V

SHARED-SUBSET SUMMARY ON THE 732 SAMPLES COMPLETED BY ALL THREE MODELS IN BOTH PROMPT SETTINGS. "MEDAPE" IS THE MEDIAN ABSOLUTE PERCENT ERROR ON NONZERO CASES. "BALACC" IS BALANCED ACCURACY ON NONZERO BB/CB TRIAGE. MEDIAN COST AND TIME ARE PER-QUERY VALUES ON THE SAME SHARED SUBSET.

Model	Evidence	Samples	FP16 MedAPE	FP16 BalAcc	FP32 MedAPE	FP32 BalAcc	FP64 MedAPE	FP64 BalAcc	Median $	Median s
`GPT-5.4`	*source-only*	732	100.0	0.500	59.2	0.889	74.5	0.670	0.0197	11.3
`GPT-5.4`	*source+SASS*	732	84.2	0.870	34.9	0.895	35.4	0.737	0.0315	12.3
`GPT OSS`	*source-only*	732	100.0	0.500	58.9	0.875	62.4	0.690	0.0006	33.2
`GPT OSS`	*source+SASS*	732	100.0	0.583	61.4	0.865	58.4	0.652	0.0008	32.5
`Opus 4.6`	*source-only*	732	100.0	0.500	43.9	0.940	52.0	0.755	0.0679	22.5
`Opus 4.6`	*source+SASS*	732	20.4	0.984	18.3	0.967	15.3	0.765	0.1066	29.8

The failure is half precision. From source alone, every model—including the frontier ones—sits exactly at the 0.500 chance baseline for FP16 (Table V), and compute-bound recall is zero (Appendix Table XI). This is a clean split: the same models that classify single-precision regimes well are no better than a coin flip on half precision when given only source.

Practical implication. A source-only triage tool can already be trusted for single-precision regime calls but must abstain—or escalate to more evidence—on half precision.

B. RQ2: What Does Cross-Compiled SASS Add, and Why?

Answer. It closes the half-precision gap, and the mechanism is compiler-synthesized arithmetic.

Adding SASS lifts FP16 BB/CB balanced accuracy from the 0.500 source-only baseline to 0.984 for `Opus 4.6` and 0.870 for `GPT-5.4` (Table V). The gain is specific to the evidence, not the model: the same frontier models were at chance for FP16 with source, while `GPT OSS`—which makes weaker use of long SASS prompts—only reaches 0.583. A paired McNemar test on the shared FP16 samples confirms the source-to-SASS classification gain is decisive for the frontier models ($p < 10^{-5}$ for `Opus 4.6`, $p < 0.01$ for `GPT-5.4`).

The reason is mechanistic and visible in the released samples. Half-precision throughput on these GPUs comes from packed `HFMA2` instructions that the compiler synthesizes; they are not syntactically present in source, so source-only reasoning conservatively misses them. In `accuracy-cuda` on the A100, source-only `Opus 4.6` predicts zero FP16 work and gets the regime wrong, while with SASS it attributes the nonzero FP16 signal to an executed `HFMA2.MMA` instruction and classifies correctly (Table XV). This is why the result generalizes beyond a single model: *half-precision Roofline reasoning requires compilation, not execution*, and cross-compilation supplies it without the target GPU.

SASS helps single precision much more modestly. FP32 balanced accuracy rises from 0.940 to 0.967 for `Opus 4.6`, and FP64 from 0.755 to 0.765—useful but far smaller than the FP16 jump, because single-precision arithmetic is already mostly source-visible.

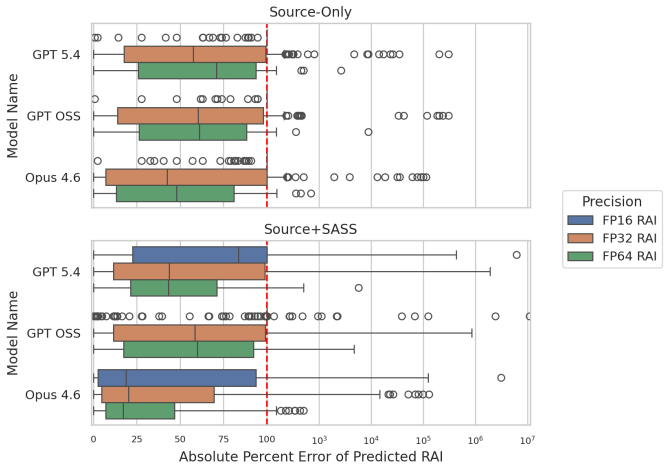

Fig. 4. Absolute percent error on nonzero RAI cases. The distribution has both a useful low-error region and a long failure tail. *source+SASS* sharply expands the low-error FP16 region for the frontier models and tightens many FP32/FP64 distributions, mirroring the classification gains.

a) Numeric RAI as a diagnostic.: The numeric error tracks the same evidence boundary, which is why we read it as a diagnostic rather than a headline. Table VI reports how often the predicted numeric RAI lands within a given relative error. Where classification is strong, the number is also close: with SASS, `Opus 4.6` is within 25% of the profiled RAI on 47.0% of nonzero FP16, 53.0% of FP32, and 44.0% of FP64 samples, with median errors of 20.4%, 18.3%, and 15.3%. Where source is blind (FP16), numeric accuracy collapses to under 1% of samples within 25%. The numeric distributions are heavy-tailed (Figure 4): some kernels are predicted almost exactly while others are orders of magnitude off, so a deployed tool should treat numeric RAI as a coarse hint and rely on the more robust regime classification.

C. RQ3: How Do These Signals Compare in Accuracy and Cost?

Answer. The LLM signals beat lightweight static baselines in most regimes, they form a clear cost/accuracy tier, and they are far cheaper than profiling.

a) Versus static baselines.: Table VII compares the best off-the-shelf LLM against the best static reference for

TABLE VI

NONZERO RAI PREDICTION SUCCESS RATES ON THE 732-SAMPLE SHARED SUBSET. A ROW IS COUNTED AS ACCURATE AT THRESHOLD x WHEN ITS ABSOLUTE PERCENT ERROR IS AT MOST x. THE 25% COLUMN IS THE MAIN SUCCESS-ENVELOPE THRESHOLD; 10% AND 50% SHOW STRICTER AND LOOSER VIEWS OF THE SAME DISTRIBUTION.

Model	Evidence	Precision	Nonzero samples	≤10%	≤25%	≤50%	MedAPE
`GPT-5.4`	*source-only*	FP16	219	0.9	0.9	2.3	100.0
`GPT-5.4`	*source-only*	FP32	236	19.1	30.9	46.6	59.2
`GPT-5.4`	*source-only*	FP64	109	13.8	20.2	29.4	74.5
`GPT-5.4`	*source+SASS*	FP16	219	18.3	24.7	32.9	84.2
`GPT-5.4`	*source+SASS*	FP32	236	24.2	42.4	58.5	34.9
`GPT-5.4`	*source+SASS*	FP64	109	11.0	26.6	51.4	35.4
`GPT OSS`	*source-only*	FP16	219	0.5	0.5	1.4	100.0
`GPT OSS`	*source-only*	FP32	236	19.5	30.9	44.1	58.9
`GPT OSS`	*source-only*	FP64	109	12.8	19.3	31.2	62.4
`GPT OSS`	*source+SASS*	FP16	219	3.7	8.2	9.6	100.0
`GPT OSS`	*source+SASS*	FP32	236	22.9	30.5	44.1	61.4
`GPT OSS`	*source+SASS*	FP64	109	12.8	23.9	33.0	58.4
`Opus 4.6`	*source-only*	FP16	219	0.5	0.5	2.3	100.0
`Opus 4.6`	*source-only*	FP32	236	25.8	36.0	52.1	43.9
`Opus 4.6`	*source-only*	FP64	109	13.8	28.4	36.7	52.0
`Opus 4.6`	*source+SASS*	FP16	219	38.4	47.0	55.7	20.4
`Opus 4.6`	*source+SASS*	FP32	236	31.4	53.0	69.5	18.3
`Opus 4.6`	*source+SASS*	FP64	109	25.7	44.0	63.3	15.3

TABLE VII

BEST-OFF-THE-SHELF LLM VERSUS BEST STATIC REFERENCE FOR TASK C NONZERO BB/CB TRIAGE ON THE SHARED SUBSET. EACH CELL REPORTS BALANCED ACCURACY AND NONZERO MEDAPE FOR THE STRONGEST METHOD IN THAT EVIDENCE FAMILY, SELECTED SEPARATELY WITHIN EACH PROMPT/PRECISION REGIME. THE DISCORDANT-PAIR COLUMNS REPORT EXACT PAIRED WINS FOR TASK C CORRECTNESS ON THE SAME SAMPLES.

Evidence	Precision	Best LLM	Task C / MedAPE	Best static	Task C / MedAPE	Discordant wins LLM/static (p)
source-only	FP16	`GPT-5.4`	0.500 / 100.0	Source learned ET	0.723 / 217.7	10/17 (0.248)
source-only	FP32	`Opus 4.6`	0.940 / 43.9	Source learned RF	0.750 / 100.0	52/3 (¡0.001)
source-only	FP64	`Opus 4.6`	0.755 / 52.0	Source lexical	0.683 / 99.5	21/13 (0.229)
source+SASS	FP16	`Opus 4.6`	0.984 / 20.4	SASS learned ET	0.608 / 163.1	33/5 (¡0.001)
source+SASS	FP32	`Opus 4.6`	0.967 / 18.3	SASS learned ET	0.730 / 100.0	57/4 (¡0.001)
source+SASS	FP64	`Opus 4.6`	0.765 / 15.3	SASS learned RF	0.841 / 62.4	5/13 (0.096)

nonzero BB/CB classification. The LLM wins decisively (exact paired tests, $p < 0.001$) in source-only FP32, *source+SASS* FP16, and *source+SASS* FP32. The static side is real in two regimes. In source-only FP16, where every LLM is at chance, a grouped source-feature baseline reaches 0.723; and in *source+SASS* FP64, a learned SASS baseline reaches 0.841 versus 0.765 for the best LLM (a point-estimate advantage, not a decisive paired win). Numeric RAI estimation, by contrast, stays a clean LLM separator: even where a static baseline is competitive for classification, its nonzero MedAPE remains far larger than the strongest LLM (for *source+SASS* FP64, 62.4% for the best static baseline versus 15.3% for `Opus 4.6`). This is why we frame deployment as a cascade rather than an LLM-only tool: cheap static features should handle source-only FP16 and *source+SASS* FP64, and LLMs should handle the regimes where they win.

b) Cost tiering.: Table V also reports median per-query cost and latency, and the configurations form a tier. The budget `GPT OSS` classifies the source-only FP32 regime at 0.875 balanced accuracy for a median $0.0006 per query; `GPT-5.4` reaches 0.889 for $0.0197; and `Opus 4.6` reaches 0.940 for $0.0679. For FP16, only the SASS-backed frontier tier works, at $0.0315 (`GPT-5.4`) to $0.1066 (`Opus 4.6`) per query. Even the most expensive configuration is roughly $0.11 per kernel-GPU pair. The recurring objection from performance-modeling venues—that LLM calls might cost as much as just measuring—does not hold for the source-only single-precision tier: at $0.0006 per kernel it is well below the marginal cost of a full compile-and-profile pass and, unlike profiling, needs no target-hardware access at all. The frontier+SASS tier is more nuanced and we treat it honestly; Section VI works the comparison through.

c) Profiling-priority corollary.: The same classification signal can also order a scarce profiling queue. Table IX ranks samples by predicted RAI relative to the balance point and profiles only the top 10%. *source+SASS* `GPT-5.4` and `Opus 4.6` surface every FP16 compute-bound kernel at that budget, and source-only FP32 rankings surface 82–96% of true compute-bound kernels; FP64 plateaus near 0.55–0.59. Because compute-bound kernels are rare in this corpus (only 34 of 732 FP16 samples; Table II), this 10% budget exceeds the positive count, so it is a high-recall but low-precision operating point. We therefore report it as a corollary of the classification result rather than as a separate utility claim, and the appendix budget sweep shows the ranking story is broadly stable across 5/10/20/30% budgets while also flagging where static rankers win.

TABLE VIII
EMPIRICAL SUCCESS ENVELOPE FOR OFF-THE-SHELF RAI PREDICTION IN THIS BENCHMARK.

Regime	Evidence pattern	What works	Main caution
Works well	*source+SASS* FP16/FP32 with lowered arithmetic visible	`Opus 4.6` reaches 47.0%/53.0% of nonzero samples within 25% error and 0.984/0.967 BB/CB BalAcc.	Requires compilation to SASS for the target architecture.
Works well	*source-only* FP32 with source-visible loop and memory structure	`Opus 4.6` is within 25% error on 36.0% of nonzero samples and reaches 0.940 BB/CB BalAcc.	DRAM-visible traffic can still break source-level reasoning.
Mixed	*source-only* FP64 and *source+SASS* FP64	Many individual samples are accurate; static SASS baselines can match or beat LLM triage in aggregate.	Treat LLM output as one signal in a static-analysis cascade.
Poor	*source-only* FP16	Nearly all nonzero samples miss the 25% threshold and BB/CB triage stays at the trivial baseline.	Use SASS or cheap static filters before trusting numeric FP16 predictions.
Poor	Context-overflow or very long prompts	`GPT OSS` has lower completion coverage and weaker SASS use.	Coverage must be reported, and incomplete samples should not be hidden in aggregate metrics.

TABLE IX
COMPUTE-BOUND RECALL WHEN PROFILING ONLY THE TOP 10% OF SAMPLES RANKED BY PREDICTED RAI RELATIVE TO THE GPU BALANCE POINT ON THE 732-SAMPLE SHARED SUBSET. EACH PRECISION BUDGETS 74 OF 732 SAMPLES. RANDOM RANKING WOULD RECOVER 10% BY CONSTRUCTION.

Model	Evidence	FP16 R@10%	FP32 R@10%	FP64 R@10%
`GPT-5.4`	*source-only*	0.000	0.820	0.566
`GPT-5.4`	*source+SASS*	1.000	0.820	0.585
`GPT OSS`	*source-only*	0.000	0.820	0.547
`GPT OSS`	*source+SASS*	0.176	0.820	0.566
`Opus 4.6`	*source-only*	0.000	0.960	0.585
`Opus 4.6`	*source+SASS*	1.000	0.940	0.566

VI. DISCUSSION

a) What this enables for AI-assisted software engineering.: The strongest positive result is that off-the-shelf LLMs can classify a kernel's BB/CB Roofline regime from software artifacts alone, without ever touching the target accelerator. That is directly relevant to AI coding agents and developer-facing tools. Before spending target-GPU time, an agent can ask a narrower question than "will this be faster?": is this kernel limited by data movement or by computation on the target device, and is the available evidence enough to answer that yet? The answer is evidence-dependent. For single precision, source artifacts are already enough for a confident regime call. For half precision, source is blind and the agent must obtain cross-compiled SASS first—but that is still a hardware-free step. Numeric RAI serves as a secondary diagnostic: when it is also accurate, it explains and reinforces the regime call; when it is in the heavy-tailed failure region, it signals that the sample should be escalated to profiling.

b) Where source-only and source+SASS fit in a real workflow.: The paper also clarifies that execution-free reasoning has two meaningful stages. *source-only* fits pre-compilation tasks such as code review, design exploration, and agent planning. *source+SASS* fits a later stage where the code can be compiled for a target architecture but the accelerator is still unavailable for execution or profiling. Treating both stages as the same "hardware-free" scenario obscures an important software-engineering distinction: compilation artifacts are themselves a development resource.

c) Cost relative to profiling.: A recurring objection to LLM-based performance reasoning is that API calls may cost as much as simply measuring on a GPU. Our per-query costs let us examine this directly, and the answer differs sharply by tier. The source-only single-precision tier is effectively free: a `GPT OSS` regime classification costs a median $0.0006 per kernel-GPU pair, so triaging the entire 732-sample shared subset costs under $0.50. No profiling run approaches that, and—more importantly for the motivating scenario—the LLM path consumes no scarce target-hardware time, whereas profiling requires getting a kernel onto the contended accelerator at all. The frontier+SASS tier is the honest edge case: at roughly $0.11 per kernel-GPU pair for `Opus 4.6`, its dollar cost is comparable to the marginal cost of a short profiling run at public cloud GPU rates (a few dollars per GPU-hour), so it is not justified by dollar savings. Its justification is instead *access and timing*: it produces a half-precision regime call when the target GPU is unavailable, rationed, or not yet provisioned, using only a cross-compile step. The exact crossover depends on profiling wall-clock, which is workload- and counter-set-dependent; the qualitative conclusion—cheap tier far below profiling, premium tier comparable in dollars but hardware-free—does not.

d) Why deterministic and learned static features still matter.: The added reference baselines sharpen the methodological boundary. A simple lexical source rule is not enough to explain the useful FP32/FP64 source-only signal, and a runtime×GPU median baseline collapses outside the zero-heavy FP16 split. But supervised source-only baselines can still recover corpus-level triage cues, and architecture-specific SASS counts already expose substantial post-compilation structure after lowering. In a few regimes, especially source-only FP16 queueing and *source+SASS* FP64 triage, the static baselines are competitive with or stronger than one-shot prompting. That motivates a *hypothesis* we do not yet validate end-

TABLE X
DEPLOYMENT ROUTING SUMMARY FROM THE SHARED SUBSET. THESE ARE CORPUS-BACKED FIRST-STAGE CHOICES, NOT UNIVERSAL RULES.

Regime	First stage	Why
source-only FP16	Static-first	One-shot LLMs stay at the trivial Task C baseline; cheap source features recover more queueing signal.
source-only FP32/FP64	Frontier LLM	Best pre-compilation triage overall, especially for FP32 where paired LLM wins are decisive.
source+SASS FP16/FP32	Frontier LLM	Largest paired gains once lowered arithmetic and DRAM-visible behavior become explicit.
source+SASS FP64	Static-first, LLM optional	Cheap SASS counts and learned SASS features already match or beat one-shot LLM triage in this regime.

to-end: a cascade that combines deterministic compiler or disassembly features, lightweight supervised models, and LLM reasoning could outperform any single strategy, routing each evidence tier to the method that wins it. Table X records the per-regime first-stage choice the released evidence supports; we present it as a routing hypothesis, not a measured system, because we have not built or evaluated the combined cascade and because choosing a route presumes the precision and evidence tier are known in advance.

e) What the released samples look like in practice.: The released samples make the aggregate story concrete. In `accuracy-cuda`, source-only `Opus 4.6` conservatively predicts zero FP16 work because compiler-generated `HFMA2` materialization is not justified from source alone; with SASS, the same model attributes the nonzero FP16 signal to an executed `HFMA2.MMA` instruction and matches the profiler almost exactly. In `boxfilter-cuda`, the source already exposes the 21-tap loop trip count, the 64 output threads per block, and the roughly 1 MiB read footprint, so source-only FP32 reasoning is effectively exact.

f) What tool builders should and should not take from the results.: Tool builders should use the method as a triage filter, not as a ground-truth oracle. The useful deployment is to prioritize profiling effort, annotate likely memory-side versus compute-side cases, or flag kernels whose source-level reasoning is too uncertain to trust. The unsafe deployment is to use the predictions as a runtime substitute, a speedup estimator, or a universal static analysis of GPU performance. The FP16 results make that limit especially clear: without post-compilation evidence, the models often miss work that only becomes visible after lowering.

g) What the exploratory failure map still suggests.: The generated feature-vote artifacts remain exploratory because their labels come from an auxiliary LLM-voting pipeline rather than human annotation. Their descriptive pattern is consistent with the comparative results above: division, special math, reusable subexpressions, and loop-invariant floating-point work remain difficult largely because source-visible intent is a weak proxy for compiled arithmetic and DRAM-visible traffic in exactly those cases.

h) Why the benchmark release matters.: Even with the method's limitations, the released corpus is a useful research contribution in its own right. It exposes a benchmarkable software-engineering task with source files, build context, runtime arguments, SASS, profiler-derived labels, and model outputs. The added deterministic and learned reference baselines show why that matters: the benchmark is not just for ranking off-the-shelf LLMs, but for comparing multiple execution-free reasoning strategies that win in different regimes. It still supports several follow-on directions that require new experiments: stronger compiler-grade analyzers, repeated-query robustness studies, and hybrid predictors that combine compiler features with LLM reasoning.

i) Why the claims are intentionally narrow.: The measured evidence supports execution-free triage, not general GPU-performance prediction. That narrower claim is still valuable: it better matches the observed behavior, it better fits the needs of AI-assisted software engineering, and it gives future work a cleaner benchmark target.

VII. THREATS TO VALIDITY AND LIMITATIONS

a) Benchmark representativeness.: The corpus comes from HeCBench and covers only the 95 benchmark programs that yielded the released 254-kernel evaluation subset. That is a meaningful but still selective sample of CUDA and OpenMP GPU software. The results should therefore be read as evidence about this benchmarked task, not as a claim about all GPU kernels or all accelerator software.

b) Hardware scope.: We study only NVIDIA GPUs, and only four of them: RTX 3080, A10, A100, and H100. The architectural generalization claims are therefore limited to those device families and their balance-point regimes. The benchmark does not cover AMD or Intel GPUs, CPU-only kernels, or other accelerator programming models such as HIP and SYCL.

c) Task scope.: The prediction target is deliberately narrow. We predict only the first kernel invocation, only DRAM-level RAI, and only the per-precision FP16/FP32/FP64 decomposition used in this study. We do not predict runtime, throughput, speedup, cache-aware arithmetic intensity, or end-to-end application performance. Readers should therefore resist extrapolating the results beyond early first-invocation triage.

d) Ground-truth measurement.: Ground truth comes from profiler counters, not from source-level reasoning. That is the right choice for our task, but it means the denominator is DRAM-visible traffic rather than all source-visible loads and stores. Cache write-back behavior, replay effects, and profiler noise can therefore influence the target quantity. We reduce noise by averaging three

first-invocation profiles, but this does not eliminate all measurement uncertainty.

e) Prompting and model stability.: The released evaluation subset queries the expensive models once per completed sample. We therefore cannot estimate run-to-run stochastic variability from the existing artifact, and closed-model behavior may drift across snapshots. We address this by narrowing claims and by reporting completed-sample counts explicitly, but we cannot retroactively add repeated-query evidence.

f) Coverage imbalance.: `GPT OSS` does not complete the full evaluation subset: it has 858 source-only and 845 *source+SASS* completed samples instead of 1,016. The missing samples are primarily context-overflow or completion failures on long prompts, which is part of the deployment cost of cheaper long-context configurations. Cross-model comparisons are therefore restricted to the 732 samples completed by all three models in both prompt settings. This is a fairer comparison, but it also means that the cheapest model's reported quality is conditioned on the samples where it completed.

g) Baseline limitations.: We now add context-only, deterministic, and lightweight learned tree baselines, and the learned baselines remain stable across five seeds. But those baselines are still deliberately modest. The deterministic baselines use approximate lexical and instruction-mix counts rather than compiler-grade analyzers. The learned baselines are evaluated with grouped cross-validation on the released corpus rather than on a separately curated external training/test split. Stronger compiler-driven or larger-scale learned baselines could therefore do better than the references reported here.

h) Failure-analysis validity.: The feature/error analysis is exploratory because its source-feature labels come from an auxiliary LLM-voting pipeline, not human annotation. We therefore treat the appendix heatmap and its prevalence/BH-corrected sanity check only as descriptive evidence about recurring error patterns. It should not be interpreted as statistically validated causal evidence.

i) Possible benchmark leakage.: Because the models are off-the-shelf and trained on large web corpora, some kernels or their close variants may have appeared in pretraining data. We cannot rule out such contamination completely. That is another reason to frame the result as an empirical benchmark study rather than a claim about intrinsic reasoning ability in isolation.

VIII. Conclusion

We framed LLM-based GPU performance reasoning as an ICSE problem: deciding a kernel's BB/CB Roofline regime from software artifacts, before any profiling run, for developers and coding agents that cannot reach the target accelerator. We can now answer the three research questions directly.

RQ1 (source-only classification). Off-the-shelf LLMs classify the single-precision Roofline regime well from source alone—`Opus 4.6` reaches 0.940 balanced accuracy for FP32 and 0.755 for FP64, and even the budget `GPT OSS` reaches 0.875 for FP32—but they are no better than chance for half precision (0.500 for all three models).

RQ2 (what SASS adds, and why). Cross-compiled SASS closes the half-precision gap, lifting FP16 balanced accuracy from 0.500 to 0.984 (`Opus 4.6`) and 0.870 (`GPT-5.4`). The cause is mechanistic: the decisive FP16 arithmetic is compiler-synthesized `HFMA2` work that is invisible in source. The actionable form of this finding is that half-precision Roofline reasoning requires *compilation*, not execution, and compilation needs no target hardware.

RQ3 (accuracy and cost). The LLM signals beat lightweight static baselines in most regimes and form a clear cost tier: source-only single-precision classification is near-free ($0.0006 per kernel) and hardware-free, while the frontier+SASS tier costs about $0.11 per kernel and is justified by access and timing rather than dollar savings. Static baselines win in source-only FP16 and *source+SASS* FP64, which is why we frame deployment as a cost-tiered cascade rather than an LLM-only oracle.

The evidence does not support a stronger claim. These models do not replace profilers, they do not predict runtime in general, numeric RAI is reliable only in identifiable regimes, and the cascade remains a routing hypothesis we have not yet validated end-to-end. What we show is narrower and useful: execution-free Roofline-regime classification is a benchmarkable software-engineering task, off-the-shelf prompting already solves it well in identifiable, cheaply-detectable evidence tiers, and the source-versus-compilation boundary—not model choice—is what governs the hardest case.

References

[1] H. Li, H. Zhang, and A. E. Hassan, "The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering," Jul. 2025, arXiv:2507.15003 [cs]. [Online]. Available: http://arxiv.org/abs/2507.15003

[2] J. Gong, V. Voskanyan, P. Brookes, F. Wu, W. Jie, J. Xu, R. Giavrimis, M. Basios, L. Kanthan, and Z. Wang, "Language Models for Code Optimization: Survey, Challenges and Future Directions," Jan. 2025, arXiv:2501.01277 [cs]. [Online]. Available: http://arxiv.org/abs/2501.01277

[3] A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, and A. Mirhoseini, "KernelBench: Can LLMs Write Efficient GPU Kernels?" Feb. 2025, arXiv:2502.10517 [cs]. [Online]. Available: http://arxiv.org/abs/2502.10517

[4] W. Chen, J. Zhu, Q. Fan, Y. Ma, and A. Zou, "CUDA-LLM: LLMs Can Write Efficient CUDA Kernels," Jun. 2025, arXiv:2506.09092 [cs]. [Online]. Available: http://arxiv.org/abs/2506.09092

[5] S. Williams, A. Waterman, and D. A. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Communications of The ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009, mAG ID: 2002555321.

[6] S. Hong and H. Kim, "An analytical model for a GPU architecture with memory-level and thread-level parallelism awareness," *SIGARCH Comput. Archit. News*, vol. 37, no. 3, pp. 152–163, Jun. 2009. [Online]. Available: https://dl.acm.org/doi/10.1145/1555815.1555775

[7] S. S. Baghsorkhi, M. Delahaye, S. J. Patel, W. D. Gropp, and W.-m. W. Hwu, "An adaptive performance modeling tool for GPU architectures," in *Proceedings of the 15th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP '10. New York, NY, USA: Association for Computing Machinery, Jan. 2010, pp. 105–114. [Online]. Available: https://dl.acm.org/doi/10.1145/1693453.1693470

[8] G. Alavani, K. Varma, and S. Sarkar, "Predicting Execution Time of CUDA Kernel Using Static Analysis," in *2018 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCloud/SocialCom/SustainCom)*. Piscataway, NJ, USA: IEEE, Dec. 2018, pp. 948–955. [Online]. Available: https://ieeexplore.ieee.org/document/8672234/

[9] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás, "GPU Static Modeling Using PTX and Deep Structured Learning," *IEEE Access*, vol. 7, pp. 159 150–159 161, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8890640/

[10] L. Braun, S. Nikas, C. Song, V. Heuveline, and H. Fröning, "A Simple Model for Portable and Fast Prediction of Execution Time and Power Consumption of GPU Kernels," *ACM Trans. Archit. Code Optim.*, vol. 18, no. 1, pp. 7:1–7:25, Dec. 2021. [Online]. Available: https://dl.acm.org/doi/10.1145/3431731

[11] Y. Emonds, L. Braun, and H. Fröning, "CUDAsap: Statically-Determined Execution Statistics as Alternative to Execution-Based Profiling," in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. Piscataway, NJ, USA: IEEE, May 2023, pp. 119–130. [Online]. Available: https://ieeexplore.ieee.org/document/10171571/

[12] H.-Q. Tran, V.-S. Pham, T.-S. Pham, T.-D. Chu, A.-T. Mai, X. T. Nguyen, and T.-T. Dao, "Accurate Latency Predictor for Triton-Based GPU Kernels with PTX Features and XGBoost," in *2025 RIVF International Conference on Computing and Communication Technologies (RIVF)*. Piscataway, NJ, USA: IEEE, Dec. 2025, pp. 980–985, iSSN: 2473-0130. [Online]. Available: https://ieeexplore.ieee.org/document/11365185/

[13] Y. Peng, A. D. Gotmare, M. R. Lyu, C. Xiong, S. Savarese, and D. Sahoo, "PerfCodeGen: Improving Performance of LLM Generated Code with Execution Feedback," in *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. Piscataway, NJ, USA: IEEE, Apr. 2025, pp. 1–13. [Online]. Available: https://ieeexplore.ieee.org/document/11052794/

[14] M.-K. Nguyen-Nhat, H. D. N. Do, H. T. Le, and T. T. Dao, "LLMPerf: GPU Performance Modeling meets Large Language Models," in *2024 32nd International Conference on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*. Piscataway, NJ, USA: IEEE, Oct. 2024, pp. 1–8. [Online]. Available: https://ieeexplore.ieee.org/document/10786558/

[15] Z. Wang, C. Ramos, M. A. Awad, and K. Lowery, "Omniwise: Predicting GPU Kernels Performance with LLMs," Jun. 2025, arXiv:2506.20886 [cs]. [Online]. Available: http://arxiv.org/abs/2506.20886

[16] G. Bolet, G. Georgakoudis, H. Menon, K. Parasyris, N. Hasabnis, H. Estes, K. Cameron, and G. Oren, "Can Large Language Models Predict Parallel Code Performance?" in *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*. University of Notre Dame Conference Facilities Notre Dame IN USA: ACM, Jul. 2025, pp. 1–6. [Online]. Available: https://dl.acm.org/doi/10.1145/3731545.3743645

[17] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2007. [Online]. Available: https://www.pearson.com/en-us/subject-catalog/p/compilers-principles-techniques-and-tools/P200000003472

[18] IEEE Computer Society, "IEEE Standard for Floating-Point Arithmetic," *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, Jul. 2019. [Online]. Available: https://ieeexplore.ieee.org/stampPDF/getPDF.jsp

[19] C. Liu, S. Lu, W. Chen, D. Jiang, A. Svyatkovskiy, S. Fu, N. Sundaresan, and N. Duan, "Code Execution with Pre-trained Language Models," May 2023, arXiv:2305.05383 [cs]. [Online]. Available: http://arxiv.org/abs/2305.05383

[20] C. Lyu, L. Yan, R. Xing, W. Li, Y. Samih, T. Ji, and L. Wang, "Large Language Models as Code Executors: An Exploratory Study," Oct. 2024, arXiv:2410.06667 [cs]. [Online]. Available: http://arxiv.org/abs/2410.06667

[21] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code," Jun. 2024, arXiv:2403.07974 [cs]. [Online]. Available: http://arxiv.org/abs/2403.07974

[22] D. Xie, M. Zheng, X. Liu, J. Wang, C. Wang, L. Tan, and X. Zhang, "CORE: Benchmarking LLMs Code Reasoning Capabilities through Static Analysis Tasks," Jul. 2025, arXiv:2507.05269 [cs]. [Online]. Available: http://arxiv.org/abs/2507.05269

[23] S. Pyda, D. Nichols, and A. Bhatele, "The Shortcomings of Code LLMs in Modeling Code Properties," in *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*. Piscataway, NJ, USA: IEEE, May 2025, pp. 193–199. [Online]. Available: https://ieeexplore.ieee.org/document/11028359/

[24] Z. Jin and J. S. Vetter, "A Benchmark Suite for Improving Performance Portability of the SYCL Programming Model," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Piscataway, NJ, USA: IEEE, Apr. 2023, pp. 325–327. [Online]. Available: https://ieeexplore.ieee.org/document/10158214/

[25] T. Kadosh, N. Hasabnis, T. Mattson, Y. Pinter, and G. Oren, "Quantifying OpenMP: Statistical Insights into Usage and Adoption," in *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. Piscataway, NJ, USA: IEEE, Sep. 2023, pp. 1–7, iSSN: 2643-1971. [Online]. Available: https://ieeexplore.ieee.org/document/10363459/

Appendix A
Additional Stratified Error Plots

Appendix B
Compute-Bound Precision/Recall

Appendix C
Static Reference Baselines

Appendix D
Budget Sweep and Seed Sensitivity

Appendix E
Illustrative Released Rows

Appendix F
Exploratory Failure Map

Appendix G
Repeated-Query Robustness (Planned)

The released evaluation subset queries each model once per sample, so it cannot characterize run-to-run stochastic variability. We plan to close this gap with a low-cost repeated-query study rather than re-running the full grid. On a stratified panel of roughly 50 kernels chosen to span precision, error magnitude, balance-point proximity, class, and runtime (not only low-error cases), we will query `GPT OSS` 5–10 times—and a small `Opus 4.6` subset—and report the run-to-run distribution of the headline classification metric. Table XVI is the placeholder for that result; the panel-construction protocol is recorded in the project plan.

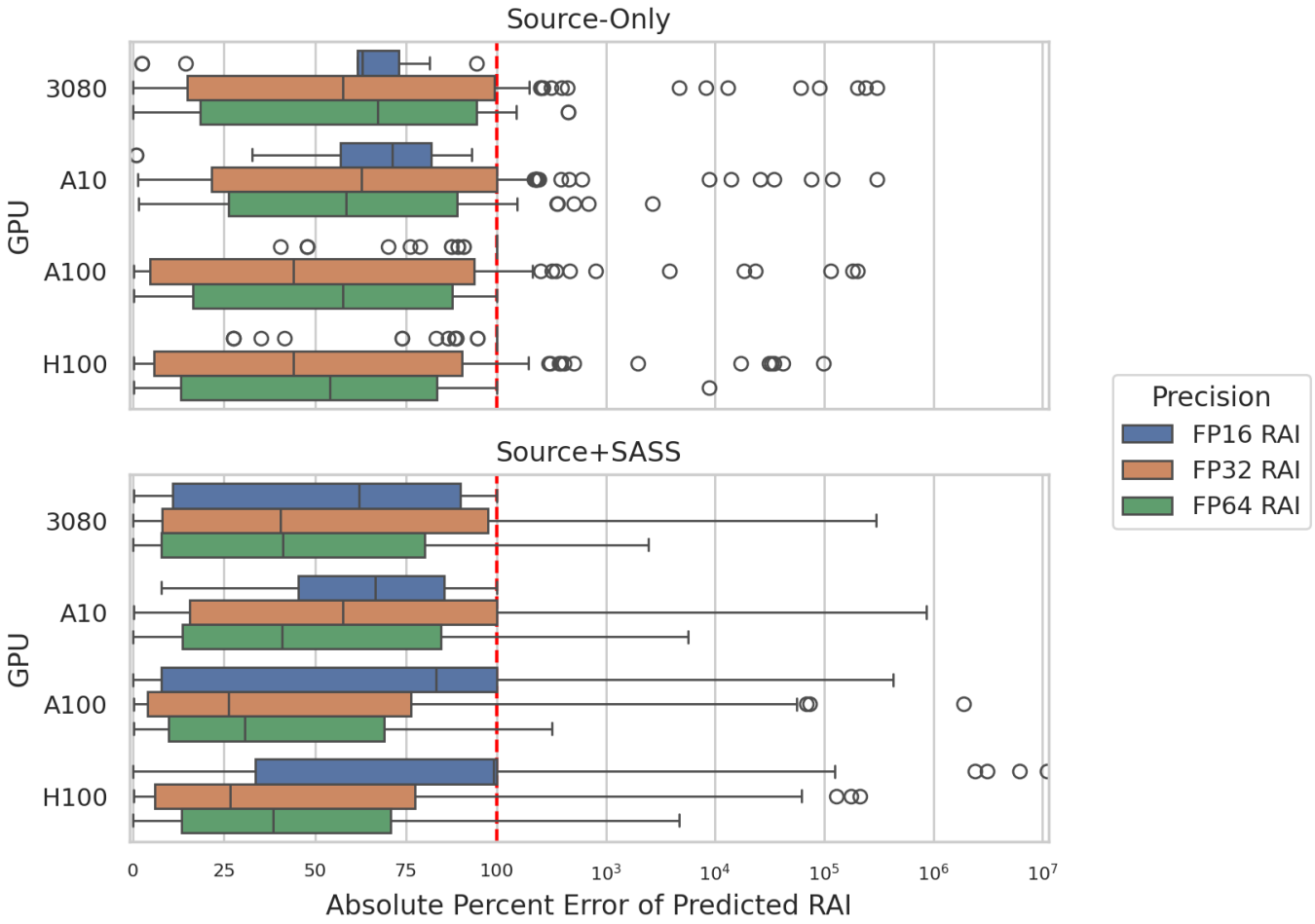

Fig. 5. Absolute percent error on nonzero RAI cases, stratified by GPU. The smaller-cache 3080/A10 settings are generally less stable than A100/H100, especially for source-level reasoning about FP32/FP64 DRAM-visible traffic.

TABLE XI
NONZERO COMPUTE-BOUND PRECISION/RECALL/F1 ON THE 732-SAMPLE SHARED SUBSET. THESE METRICS COMPLEMENT TASK C BALANCED ACCURACY BY EXPOSING WHETHER A METHOD REACHES HIGH RECALL BY OVER-PREDICTING THE COMPUTE-BOUND SIDE.

Model	Evidence	FP16 P/R/F1	FP32 P/R/F1	FP64 P/R/F1
`GPT-5.4`	*source-only*	– / 0.000 / –	0.909 / 0.800 / 0.851	0.950 / 0.358 / 0.521
`GPT-5.4`	*source+SASS*	0.730 / 0.794 / 0.761	0.952 / 0.800 / 0.870	0.903 / 0.528 / 0.667
`GPT OSS`	*source-only*	– / 0.000 / –	0.950 / 0.760 / 0.844	0.917 / 0.415 / 0.571
`GPT OSS`	*source+SASS*	0.750 / 0.176 / 0.286	0.949 / 0.740 / 0.831	0.900 / 0.340 / 0.493
`Opus 4.6`	*source-only*	– / 0.000 / –	1.000 / 0.880 / 0.936	0.966 / 0.528 / 0.683
`Opus 4.6`	*source+SASS*	0.850 / 1.000 / 0.919	0.979 / 0.940 / 0.959	0.938 / 0.566 / 0.706

TABLE XVI
[PLACEHOLDER — pending deferred experiment.]
RUN-TO-RUN STABILITY OF NONZERO BB/CB BALANCED ACCURACY ACROSS REPEATED QUERIES ON THE STRATIFIED ROBUSTNESS PANEL. EACH CELL WILL REPORT MEAN ± 95% CI OVER REPEATS. VALUES ARE TO BE FILLED FROM THE PLANNED VARIANCE STUDY.

Model	Evidence	FP16 BalAcc	FP32 BalAcc	FP64 BalAcc
`GPT OSS`	*source-only*	TBD	TBD	TBD
`GPT OSS`	*source+SASS*	TBD	TBD	TBD
`Opus 4.6`	*source-only*	TBD	TBD	TBD
`Opus 4.6`	*source+SASS*	TBD	TBD	TBD

APPENDIX H
REASONING VERSUS MEMORIZATION PROBE (PLANNED)

Because the corpus is public, some kernels may appear in pretraining data, so accurate predictions could reflect recall rather than reasoning. We plan a perturbation probe to separate the two on a panel that deliberately over-samples near-exact and well-known kernels (where memorization is the competing explanation), with high-error controls. Each kernel receives two perturbation types with opposite expectations: *cosmetic* edits (renamed identifiers, reordered functions) under which a reasoning model

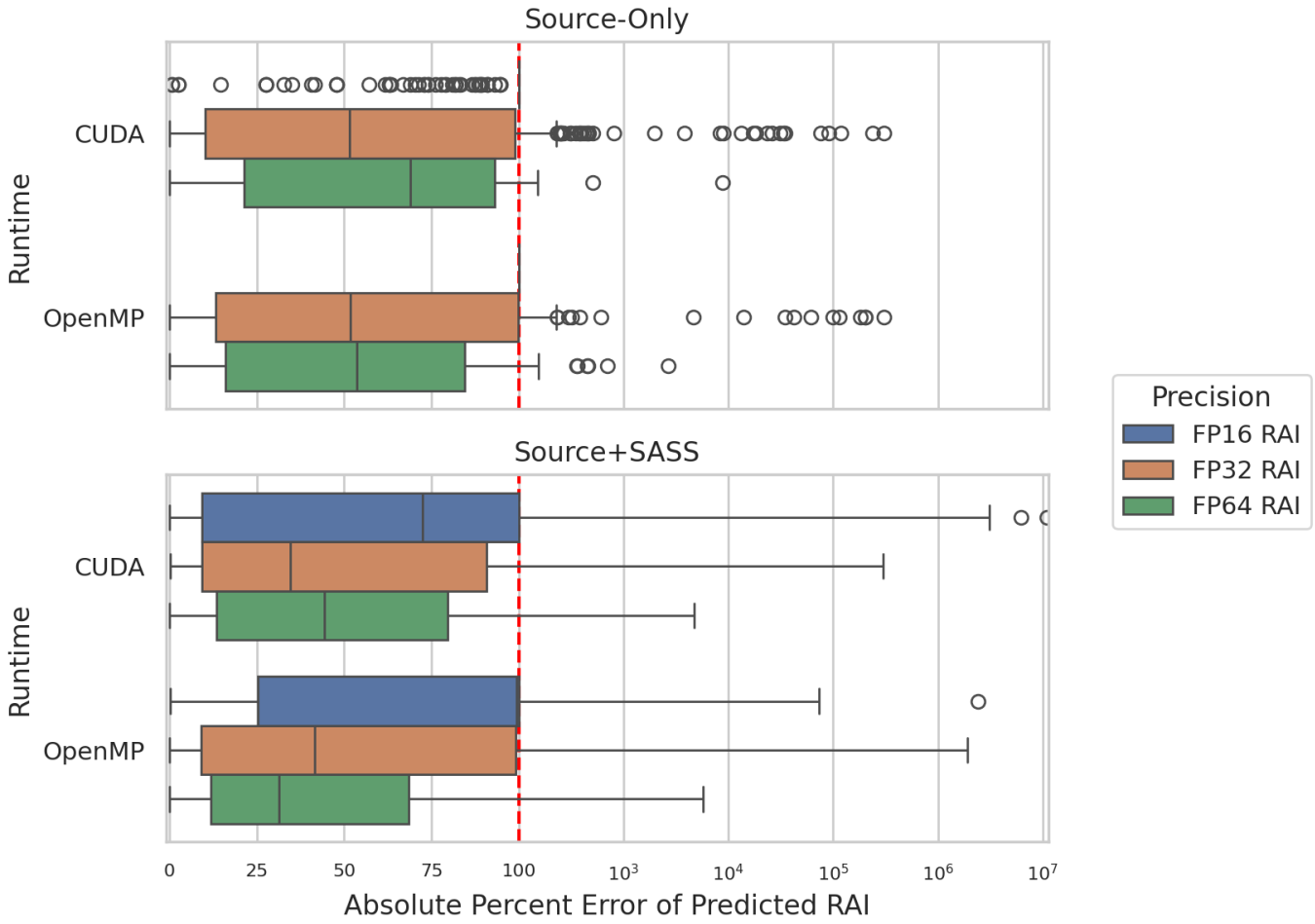

Fig. 6. Absolute percent error on nonzero RAI cases, stratified by runtime. The runtime split is precision-sensitive rather than uniform: CUDA tends to be easier for FP16/FP32 triage, while FP64 remains mixed across CUDA and OpenMP.

should be invariant, and *semantic* edits (changed loop trip counts, array dimensions, or problem-size arguments) that move the ground truth—under which a reasoning model's prediction should track the new value while a memorizing model stays anchored to the original. Where the perturbed kernel must be re-profiled to obtain new ground truth, we will use a single locally available GPU, since this probe tests the model rather than cross-device generality. We will report the correlation between predicted and true RAI change, not absolute error alone.

Appendix I
Anonymity Note

The current workspace contains public website and repository mirrors for the benchmark artifact. To preserve double anonymity, the manuscript omits those URLs. The project README records the replacement points for a de-anonymized camera-ready version.

TABLE XII

STATIC REFERENCE BASELINES ON THE 732-SAMPLE SHARED SUBSET. CONTEXT MEDIAN IS A GROUPED CROSS-VALIDATED RUNTIME×GPU BASELINE; LEXICAL AND MNEMONIC ARE DETERMINISTIC HEURISTICS; THE LEARNED BASELINES ARE FIVE-FOLD GROUPED TREE BASELINES GROUPED BY PROGRAM. FOR EACH PRECISION, THE SLASH-SEPARATED TRIPLE REPORTS TASK A ZERO/NONZERO BALANCED ACCURACY, TASK C NONZERO BB/CB BALANCED ACCURACY, AND TASK D THREE-WAY BALANCED ACCURACY. "MEDAPE" IS THE MEDIAN NONZERO ABSOLUTE PERCENT ERROR.

Baseline	FP16 A/C/D	FP32 A/C/D	FP64 A/C/D	FP16 MedAPE	FP32 MedAPE	FP64 MedAPE
Context median CV	0.823 / 0.500 / 0.549	0.500 / 0.500 / 0.333	0.500 / 0.500 / 0.333	100.0	100.0	100.0
Source lexical	0.523 / 0.500 / 0.351	0.693 / 0.489 / 0.444	0.651 / 0.683 / 0.531	100.0	99.7	99.5
SASS mnemonic	0.983 / 0.500 / 0.656	0.897 / 0.577 / 0.649	0.995 / 0.756 / 0.834	95.9	64.9	68.8
Source learned RF	0.845 / 0.712 / 0.699	0.804 / 0.750 / 0.674	0.659 / 0.644 / 0.544	100.0	100.0	100.0
Source learned ET	0.842 / 0.723 / 0.703	0.772 / 0.695 / 0.622	0.624 / 0.498 / 0.430	217.7	100.0	100.0
SASS learned RF	0.973 / 0.586 / 0.705	0.913 / 0.723 / 0.743	0.879 / 0.841 / 0.815	190.9	100.0	62.4
SASS learned ET	0.917 / 0.608 / 0.686	0.860 / 0.730 / 0.712	0.835 / 0.725 / 0.692	163.1	100.0	100.0

TABLE XIII

BUDGET-SWEEP COMPUTE-BOUND RECALL FOR THE STRONGEST SOURCE-ONLY AND SOURCE+SASS METHODS IN EACH FAMILY, SELECTED BY MEAN RECALL ACROSS THE 5/10/20/30% SWEEP ON THE SHARED SUBSET. THIS SHOWS WHETHER THE RANKING BEHIND TABLE IX IS STABLE BEYOND THE SINGLE 10% OPERATING POINT.

Precision	Method	R@5%	R@10%	R@20%	R@30%
FP16	*source-only* best LLM (Opus 4.6)	0.000	0.000	0.412	0.471
FP16	*source-only* best static (Source learned RF)	0.382	0.824	0.853	0.941
FP16	*source+SASS* best LLM (Opus 4.6)	0.971	1.000	1.000	1.000
FP16	*source+SASS* best static (SASS mnemonic)	0.294	0.706	0.971	1.000
FP32	*source-only* best LLM (Opus 4.6)	0.740	0.960	0.960	0.960
FP32	*source-only* best static (Source learned RF)	0.480	0.720	0.960	0.980
FP32	*source+SASS* best LLM (Opus 4.6)	0.660	0.940	0.940	0.940
FP32	*source+SASS* best static (SASS mnemonic)	0.440	0.760	1.000	1.000
FP64	*source-only* best LLM (Opus 4.6)	0.208	0.585	0.604	0.604
FP64	*source-only* best static (Source learned RF)	0.340	0.377	0.415	0.472
FP64	*source+SASS* best LLM (Opus 4.6)	0.208	0.566	0.566	0.566
FP64	*source+SASS* best static (SASS mnemonic)	0.604	0.811	1.000	1.000

TABLE XIV

SEED SENSITIVITY FOR THE LEARNED STATIC BASELINES ACROSS FIVE GROUPED-CV SEEDS. EACH ENTRY REPORTS MEAN±STD OVER SEEDS FOR TASK C BALANCED ACCURACY AND NONZERO MEDAPE. THE LOW SPREAD SHOWS THAT THE STATIC-BASELINE COMPARISONS ARE NOT DRIVEN BY AN UNUSUALLY FAVORABLE RANDOM SEED.

Baseline	FP16 C / MedAPE	FP32 C / MedAPE	FP64 C / MedAPE	FP16 std(C)	FP32 std(C)	FP64 std(C)
Source learned RF	0.731 / 102.1	0.801 / 100.0	0.621 / 100.0	0.025	0.028	0.021
Source learned ET	0.649 / 214.4	0.692 / 100.0	0.515 / 100.0	0.037	0.011	0.014
SASS learned RF	0.589 / 204.0	0.738 / 100.0	0.843 / 62.0	0.007	0.009	0.007
SASS learned ET	0.601 / 184.6	0.729 / 100.0	0.729 / 100.0	0.009	0.014	0.021

TABLE XV

ILLUSTRATIVE RELEASED SAMPLES USED ONLY FOR INTUITION, NOT AS ADDITIONAL EVALUATION EVIDENCE. THESE EXAMPLES SHOW ACCURATE NUMERIC RAI PREDICTIONS IN REGIMES WHERE THE RELEVANT COMPUTATION AND MEMORY BEHAVIOR ARE VISIBLE TO THE MODEL.

Case	Released sample	Expected → predicted RAI	Why it matters
Hidden lowered FP16 work	`accuracy-cuda`, A100, `Opus 4.6`	FP16 *source-only*: 0.0254 → 0.0; FP16 *source+SASS*: 0.0254 → 0.0256	The source-only explanation notes that compiler-generated `HFMA2` materialization may exist but cannot be justified from source alone, so it conservatively predicts zero FP16 work. The SASS-backed explanation instead points to an executed `HFMA2.MMA` instruction on the taken path and counts it as real FP16 arithmetic.
Source-visible FP32 work	`boxfilter-cuda`, A100, `Opus 4.6`	FP32 *source-only*: 21.975 → 22.0	The source already exposes the 21-tap loop trip count, the 64 output threads per block, and the roughly 1 MiB unique read footprint. In this regime, source-only reasoning is already enough and post-compilation evidence adds little.
Regular stencil-like FP32 work	`asmooth-cuda`, 3080, `Opus 4.6`	FP32 *source-only*: 0.8329 → 0.8331	The kernel has a regular source-visible arithmetic and memory pattern, so the model can infer a close DRAM-level ratio without SASS.
Source/SASS-visible FP64 work	`burger-cuda`, 3080, `Opus 4.6`	FP64 *source+SASS*: 2.9669 → 2.9684	The double-precision arithmetic and global-memory structure are visible enough after lowering for the SASS-backed estimate to nearly match the profiler-derived RAI.

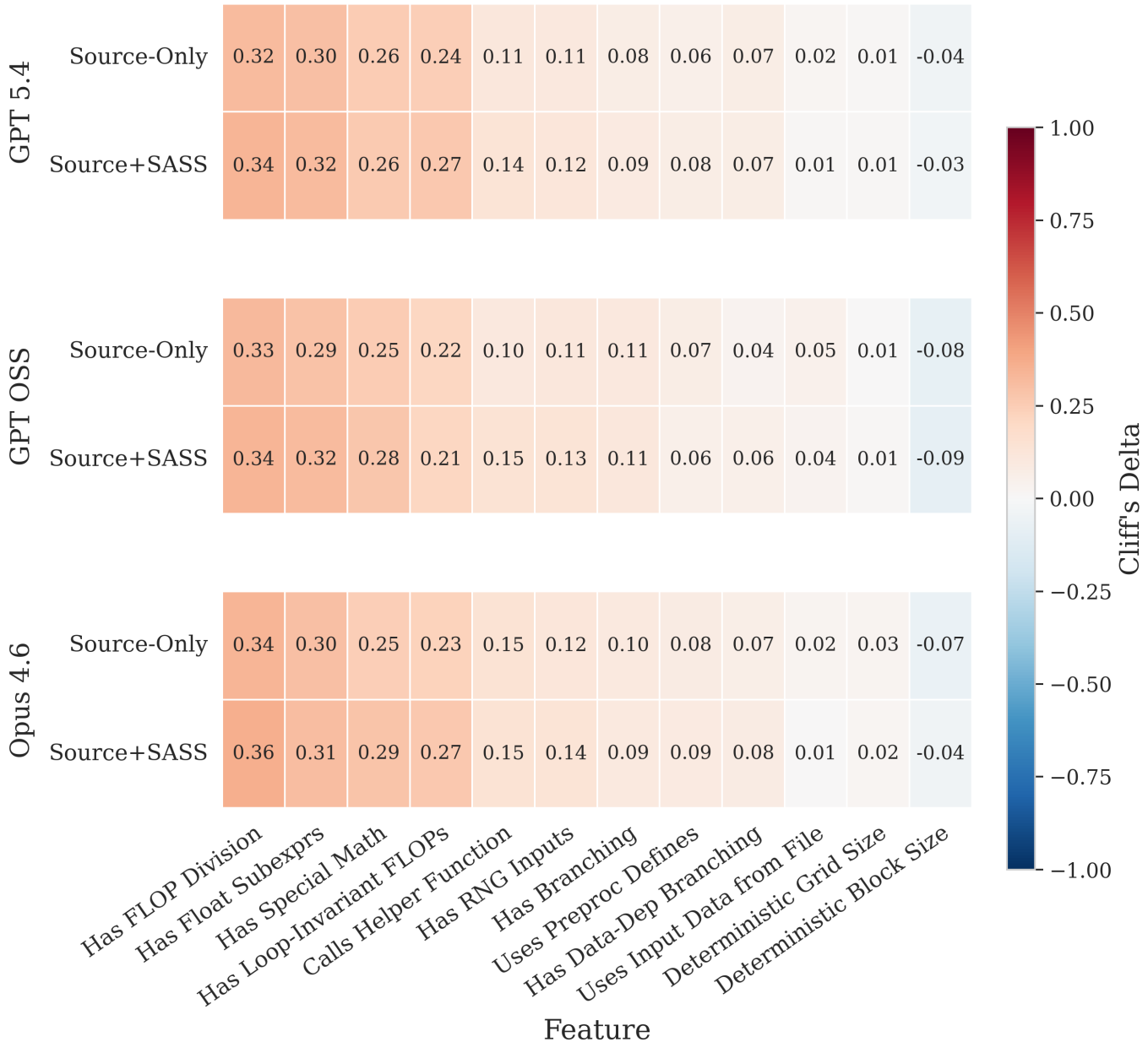

Fig. 7. Exploratory feature/error association heatmap from the released artifact. Positive Cliff's delta means higher absolute RAI error when the feature is present. The strongest repeated error signals come from division, special math, reusable subexpressions, and loop-invariant floating-point work. A post-hoc Mann–Whitney/BH sanity check across the same 72 model×prompt×feature cells preserves repeated positive associations for division, common float subexpressions, special math, loop-invariant FLOPs, helper-function calls, RNG inputs, branching, preprocessor defines, and data-dependent branching; those features appear in 7–90% of the 1,336 voted kernels. Because the feature labels are generated by an auxiliary LLM-voting pipeline, we treat this figure as descriptive rather than causal.